

**Universidad del Valle de Guatemala Programación
Orientada a Objetos
Segundo Ciclo, 2026
Catedrático: Kevin Antonio Velasquez Aguilar**



EJERCICIO 5, POLIMORFISMO

**Obed Natanael Tot Paiz
Carné:26971**

Fecha de entrega: 10/10/2026

1. Análisis del problema: Defensa de QTZ 2

Se necesita desarrollar un simulador de consola para administrar los recursos de una misión del satélite Quetzal-2. El sistema debe representar el funcionamiento de distintos módulos que colaboran en la generación de energía, la recolección de información científica y su descarga a la Tierra.

El problema consiste en coordinar estos módulos, ya que comparten recursos y dependen de su disponibilidad para operar. El orden de participación afecta los resultados: un módulo utiliza la energía y los datos que dejaron disponibles los anteriores.

1.1. Objetivo del sistema

Permitir que un operador consulte los módulos de la misión, avance la simulación por ciclos y supervise el uso de energía y la transferencia de información científica, aplicando herencia, polimorfismo y el patrón MVC.

1.2. Información que debe manejarse

Todos los módulos comparten un identificador, nombre, estado de salud, indicador de actividad y costo de construcción y además, cada tipo necesita información específica.

Tipo de módulo	Información particular
Vuelo	Tipo de instrumento cámara o sensor, cantidad de datos recolectados por ciclo, consumo de energía y ciclos efectivos de operación
Tierra	Capacidad de descarga por ciclo, energía requerida por descarga, total histórico descargado y estación a la que pertenece.
Energía	Cantidad de energía generada por ciclo y ciclos efectivos de operación.

La misión debe mantener la energía disponible, los datos pendientes de descarga en órbita y el total histórico de datos descargados, también debe poder determinarse el total histórico recolectado por cada módulo de vuelo y generado por cada módulo de energía, como sus capacidades son constantes, estos valores pueden obtenerse multiplicando su capacidad por sus ciclos efectivos de operación.

1.3. Requisitos funcionales

- Cargar una misión inicial, el programa debe iniciar con al menos diez módulos distribuidos entre los tres tipos, almacenados en una única lista polimórfica. La información inicial debe representar una misión ya en progreso y mantener coherencia entre sus recursos y acumulados.
- Listar los módulos, se deben mostrar todos los módulos construidos y sus características comunes y particulares.
- Buscar módulos, el operador debe poder consultar un módulo por su identificador o nombre y visualizar toda su información. Si no se encuentra, el sistema debe comunicarlo.
- Consultar el catálogo por costo, se deben presentar los módulos ordenados por costo de construcción, conservando el orden original de participación en los ciclos.
- Para avanzar la simulación, el sistema debe recorrer la lista en el orden establecido en la carga inicial y dar una oportunidad de operación a cada módulo. Debe mostrar el resultado de cada participación y los recursos finales de la misión.
- El informe debe incluir la cantidad total de módulos de tierra, cuántos están activos, su capacidad total de descarga activa y el total histórico descargado. También debe identificar, mediante nombre, ID y estación, todos los módulos de tierra que compartan el máximo histórico de descargas.

1.4. Reglas de funcionamiento

- Los módulos inactivos no realizan operaciones ni modifican los recursos.
- Un módulo de vuelo solo opera si dispone de toda la energía necesaria. Al operar, consume energía, agrega datos a la reserva en órbita e incrementa sus ciclos efectivos. No se permiten recolecciones parciales.
- Un módulo de tierra necesita energía suficiente y datos pendientes en órbita. Descarga la menor cantidad entre su capacidad y los datos disponibles.
- Una descarga consume toda la energía requerida, aunque se descarguen menos datos que la capacidad máxima.
- Los datos descargados se restan de la reserva en órbita y se suman al acumulado del módulo y de la misión.
- Un módulo de energía activo genera una cantidad fija de energía e incrementa sus ciclos efectivos.
- Cada operación actualiza inmediatamente los recursos compartidos.
- Si un módulo no puede operar, se informa la causa y se continúa con el siguiente.
- Los recursos y acumulados se conservan entre ciclos.

1.5. Entradas y salidas

Entradas	Salidas
Información de la carga inicial.	Listado y características de los módulos.
Opción seleccionada en el menú.	Resultado de una búsqueda.
ID o nombre que se desea buscar.	Catálogo ordenado por costo.

Solicitud de avanzar un ciclo.	Operaciones realizadas, motivos de inactividad y recursos finales.
Solicitud del resumen de comunicaciones.	Estadísticas de los módulos de tierra y sus descargas.

1.6. Restricciones y criterios de diseño

La implementación debe realizarse en Java. Los módulos deben compartir una base común mediante herencia y ejecutar su comportamiento particular mediante la sobrescritura de `procesarCiclo()`.

El patrón MVC debe separar las reglas y los datos de la misión, la interacción por consola y la coordinación de las acciones del usuario. También se requiere encapsulación, documentación interna y sobrescritura de `toString()` para representar la información de los objetos.

El enunciado no define reglas de daño, reparación o efectos de la salud sobre las operaciones y por eso se propone registrar la salud como característica informativa y utilizar el estado activo y los recursos disponibles para determinar si un módulo puede operar y tampoco se exige implementar compras, combates ni baterías.

2. Diseño Diagrama UML

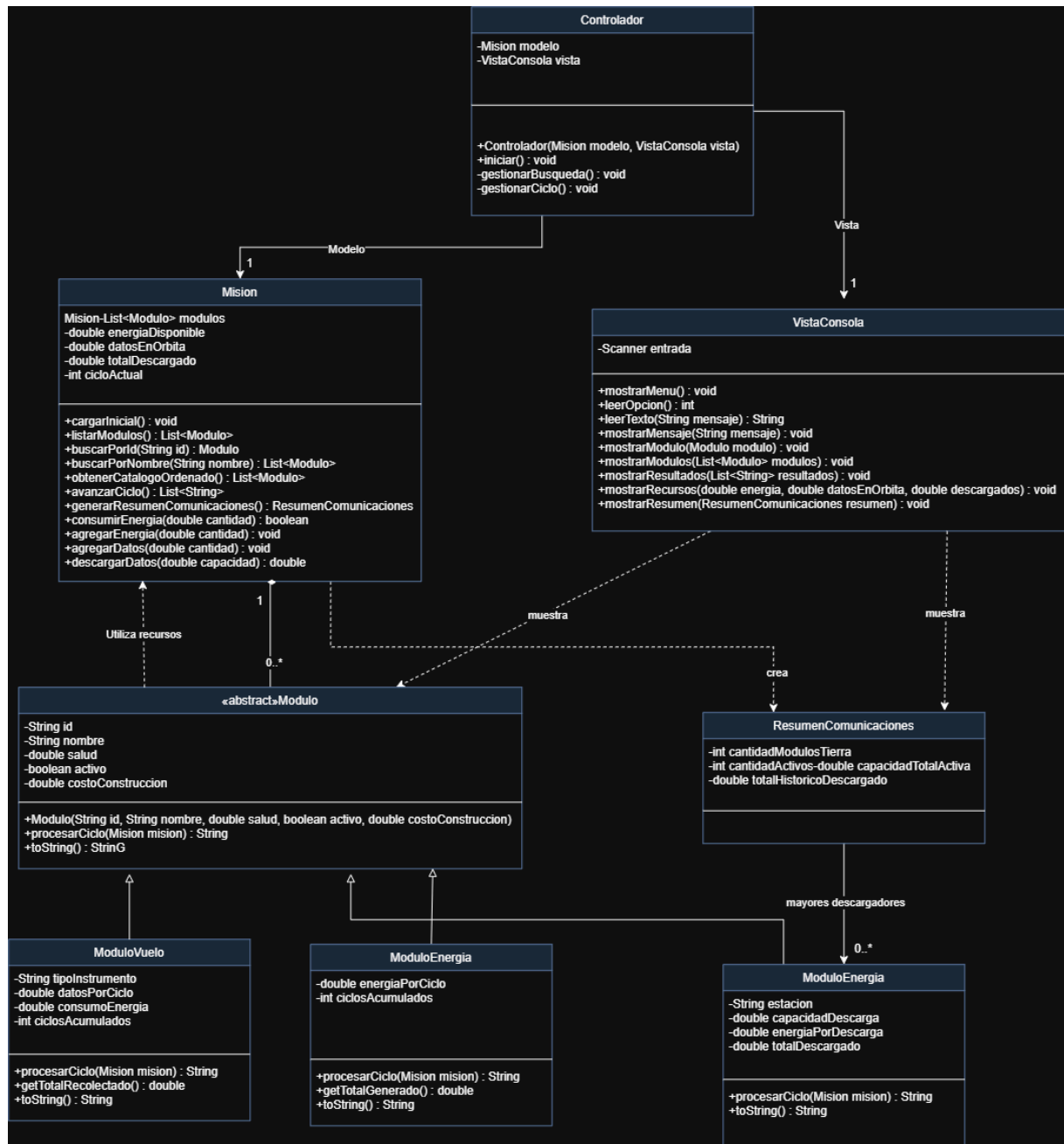
En este diseño, la clase `Modulo` reúne los atributos que comparten todos los módulos, como el ID, nombre, salud, estado y costo de construcción, las clases `ModuloVuelo`, `ModuloTierra` y `ModuloEnergia` heredan estos atributos y agregan los datos que necesitan para realizar su función.

El polimorfismo se aplica mediante el método `procesarCiclo()`. Cada tipo de módulo tiene su propia versión: los de vuelo recolectan datos, los de tierra descargan información y los de energía generan energía, esto permite guardar todos los módulos en una misma lista y llamar al mismo método para que cada uno realice la operación que le corresponde.

El programa utiliza el patrón MVC para separar las responsabilidades, el modelo guarda la información de la misión y aplica las reglas de funcionamiento, la vista muestra el menú, recibe los datos del usuario y presenta los resultados, el controlador interpreta la opción seleccionada y coordina las acciones entre el modelo y la vista.

La misión comienza con al menos diez módulos de los tres tipos y en cada ciclo, participan en el orden de la carga inicial y utilizan los recursos que dejaron disponibles los anteriores. Para mostrar el catálogo por costo, se ordena una copia de la lista, de modo que no cambie el orden de participación.

El resumen de comunicaciones reúne la información de los módulos de tierra, si varios tienen la misma cantidad máxima de datos descargados, se muestran todos los que comparten ese resultado.



3. ENLACE DE GITHUB.

<https://github.com/ntp90-cmd/Ejercicio-5-Polimorfismo-Obed.git>